NMAM INSTITUTE OF TECHNOLOGY, NITTE

Department of Information Science and Engineering

OPERATING SYSTEMS

PROGRAMMING ASSIGNMENT REPORT

Synchronization of Readers Writers and Dining Philosophers Problem using Semaphores and Monitors in Java with HTML playback of events

Submitted by

Devika K

USN: NNM24IS065

Github: https://github.com/D-evikak/OS-Task-Synchronization

Submitted To

Dr Jason Elroy Martis

Associate Professor

Department of Information Science and Engineering

Academic Year: 2026–2027

Abstract

This assignment implements the Readers-Writers and Dining Philosophers synchronization problems in Java using semaphore-based and monitor-style approaches. The programs use four readers and two writers, and four philosophers sharing four forks. Synchronization techniques ensure safe resource sharing and help prevent deadlock. A tracerecording component generates HTML simulations to visualize thread activities and execution events. The report discusses mutual exclusion, deadlock prevention, fairness, and testing results.

Keywords: Operating Systems, Java Threads, Semaphores, Monitors, Readers-Writers, Dining Philosophers, Deadlock Prevention.

1. PROBLEM STATEMENT AND SYNCHRONIZATION OBJECTIVE

Concurrent threads may access shared data or resources at the same time, causing race conditions and inconsistent results. Synchronization techniques coordinate thread execution and protect shared resources.

Readers-Writers: Multiple readers can access shared data simultaneously, but writers require exclusive access to update it. The objective is to ensure safe access to the shared value.

Dining Philosophers: Four philosophers share four forks, and each needs two adjacent forks to eat. The objective is to prevent conflicting fork usage and avoid deadlock.

The programs also record execution events and generate HTML simulations to visualize thread activities.

.

1.1 Objectives

1. Implement Readers-Writers using semaphores.
2. Implement Readers-Writers using ReentrantLock and Condition variables.
3. Implement Dining Philosophers using semaphores.
4. Implement Dining Philosophers using Java monitor mechanisms.
5. Record relevant synchronization events and generate HTML playback pages.
6. Compare mutual exclusion, deadlock prevention, and fairness considerations.

.

2. IMPLEMENTATION ENVIRONMENT

The programs are implemented in Java using concurrency features. Each reader, writer, or philosopher runs in a separate thread. Short sleep intervals make thread execution visible and do not replace synchronization mechanisms.

- **Java Threads:** Run readers, writers, and philosophers concurrently.
- **Semaphore:** Controls access to shared resources and forks.
- **ReentrantLock:** Provides explicit locking in the Readers-Writers monitor-style solution.
- **Condition:** Allows threads to wait until specific conditions are satisfied.
- **synchronized, wait(), notifyAll():** Coordinate Dining Philosophers monitor states.
- **TraceRecorder.java:** Records Dining Philosophers events and generates HTML output.
- **HTML, CSS, JavaScript:** Display recorded events through browser playback.
- **Eclipse and web browser:** Run Java programs and view HTML simulations.

3. READERS-WRITERS PROBLEM

Problem description

The shared data object contains an integer value initially set to zero. Four reader threads read the value, and two writer threads update it. Each writer performs three rounds, incrementing the shared value once per round. Therefore, when all six updates complete, the expected final value is 6.The main requirements are: multiple readers can read at the same

time; a writer must access the shared resource exclusively; and a reader must not read while a writer is updating the value.

Semaphore-based implementation

The semaphore implementation uses readerMutex to protect the activeReaders counter and resourceLock to control access to the shared resource. The first reader acquires resourceLock before entering the reading section. Other readers can then read concurrently. When the last reader exits, it releases resourceLock. Writers must acquire resourceLock before modifying the shared value.

Code Snippet 1.. Semaphore declarations for Readers-Writers

```
private static final int READER_COUNT = 4; private static
final int WRITER_COUNT = 2; private static final int
ROUNDS = 3;

private static final Semaphore readerMutex = new
Semaphore(1); private static final Semaphore
resourceLock = new Semaphore(1);

private static int activeReaders = 0; private static final
SharedData data = new SharedData();
```

The readerMutex semaphore protects changes to activeReaders. The resourceLock semaphore prevents a writer from modifying the shared value while readers are using it

Code Snippet 2. Reader entry excerpt readerMutex.acquire();

```
if (activeReaders == 0) {

resourceLock.acquire(); }

activeReaders++; readerMutex.release();

int value = data.readValue();.
```

This is a selected excerpt from the reader-entry method. The full implementation also decreases activeReaders after reading and releases resourceLock when the last reader exits.

Monitor-style implementation

The Readers-Writers monitor-style solution uses a fair ReentrantLock and two Condition objects, canRead and canWrite. It tracks activeReaders, waitingWriters, and writerActive. A reader waits while a writer is active or while writers are waiting. This writer-priority condition reduces the chance that writers will be delayed indefinitely by newly arriving readers.

Code Snippet 3. Lock and condition declarations private
static final ReentrantLock lock = new ReentrantLock(true);

```
private static final Condition canRead =
lock.newCondition(); private static final Condition canWrite
= lock.newCondition();

private static int activeReaders = 0; private static int
waitingWriters = 0; private static boolean writerActive =
false;
```

Code Snippet 4. Reader waiting condition
lock.lockInterruptibly();

```
try { while (writerActive || waitingWriters > 0)
{ canRead.await(); }

activeReaders++; entered = true;

} finally { lock.unlock() ; }
```

This is a selected excerpt from the reader-entry logic; the variable entered and the surrounding method declarations belong to the full source. Writers wait until the shared resource is no longer being used, update the value exclusively, and then signal waiting threads as appropriate.

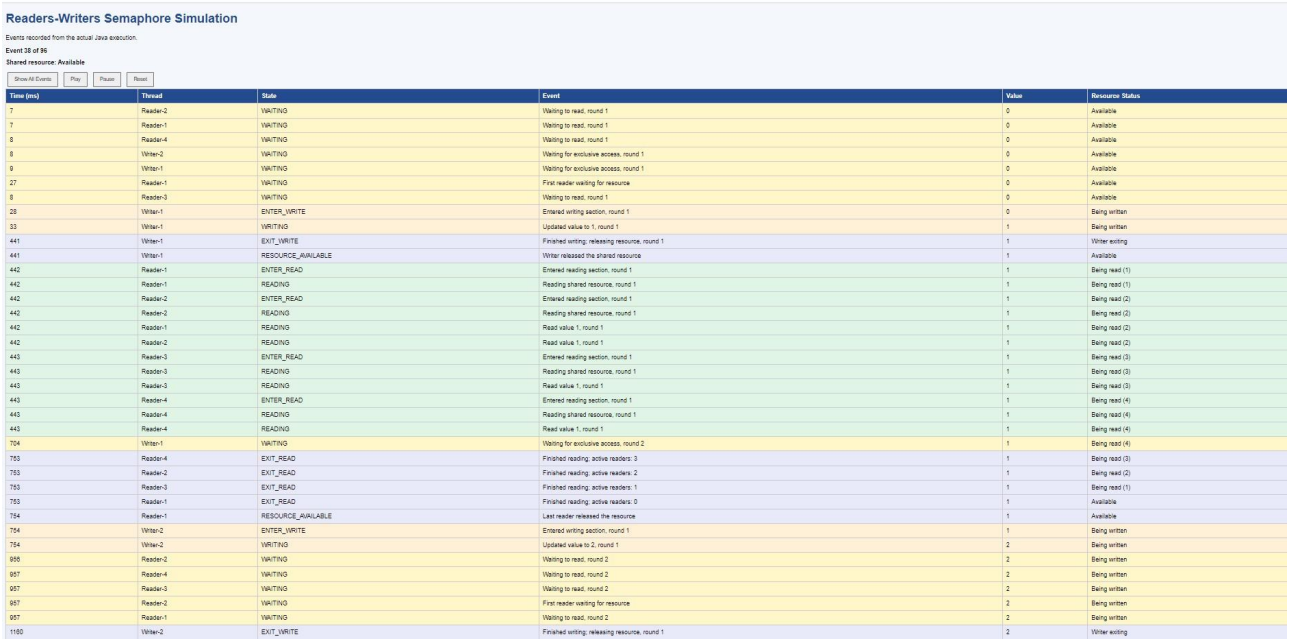
Configuration and expected result

The Readers-Writers program uses 4 reader threads and 2 writer threads, with 3 rounds per thread. The shared value starts at 0. Since each writer performs 3 updates, the expected final value is 6.

The HTML output screenshots show reader and writer events, shared-resource status, event time, and the current shared value. The captured semaphore page reports 96 events, while the monitor page reports 82 events in the shown playback state. Event counts can differ because the implementations record different event sequences.

2.1 Readers-Writers HTML output

Figure 1 shows the semaphore event playback. Waiting readers and writers are visible alongside the shared-resource status and value.



Readers-Writers Semaphore Simulation

Events recorded from the actual Java execution.

Event 38 of 96

Shared resource: Available

Show All Events | Play | Pause | Reset

Time (ms)	Thread	State	Event	Value	Resource Status
7	Reader-2	WAITING	Waiting to read, round 1	0	Available
7	Reader-1	WAITING	Waiting to read, round 1	0	Available
8	Reader-4	WAITING	Waiting to read, round 1	0	Available
8	Writer-2	WAITING	Waiting for exclusive access, round 1	0	Available
9	Writer-1	WAITING	Waiting for exclusive access, round 1	0	Available
27	Reader-1	WAITING	First reader waiting for resource	0	Available
8	Reader-3	WAITING	Waiting to read, round 1	0	Available
28	Writer-1	ENTER_WRITE	Entered writing section, round 1	0	Being written
33	Writer-1	WRITING	Updated value to 1, round 1	1	Being written
441	Writer-1	EXIT_WRITE	Finished writing; releasing resource, round 1	1	Writer exiting
441	Writer-1	RESOURCE_AVAILABLE	Writer released the shared resource	1	Available
442	Reader-1	ENTER_READ	Entered reading section, round 1	1	Being read (1)
442	Reader-1	READING	Reading shared resource, round 1	1	Being read (1)
442	Reader-2	ENTER_READ	Entered reading section, round 1	1	Being read (2)
442	Reader-2	READING	Reading shared resource, round 1	1	Being read (2)
442	Reader-1	READING	Read value 1, round 1	1	Being read (2)
442	Reader-2	READING	Read value 1, round 1	1	Being read (2)
443	Reader-3	ENTER_READ	Entered reading section, round 1	1	Being read (3)
443	Reader-3	READING	Reading shared resource, round 1	1	Being read (3)
443	Reader-3	READING	Read value 1, round 1	1	Being read (3)
443	Reader-4	ENTER_READ	Entered reading section, round 1	1	Being read (4)
443	Reader-4	READING	Reading shared resource, round 1	1	Being read (4)
443	Reader-4	READING	Read value 1, round 1	1	Being read (4)
704	Writer-1	WAITING	Waiting for exclusive access, round 2	1	Being read (4)
753	Reader-4	EXIT_READ	Finished reading; active readers: 3	1	Being read (3)
753	Reader-2	EXIT_READ	Finished reading; active readers: 2	1	Being read (2)
753	Reader-3	EXIT_READ	Finished reading; active readers: 1	1	Being read (1)
753	Reader-1	EXIT_READ	Finished reading; active readers: 0	1	Available
754	Reader-1	RESOURCE_AVAILABLE	Last reader released the resource	1	Available
754	Writer-2	ENTER_WRITE	Entered writing section, round 1	1	Being written
754	Writer-2	WRITING	Updated value to 2, round 1	2	Being written
956	Reader-2	WAITING	Waiting to read, round 2	2	Being written
957	Reader-4	WAITING	Waiting to read, round 2	2	Being written
957	Reader-3	WAITING	Waiting to read, round 2	2	Being written
957	Reader-2	WAITING	First reader waiting for resource	2	Being written
957	Reader-1	WAITING	Waiting to read, round 2	2	Being written
1160	Writer-2	EXIT_WRITE	Finished writing; releasing resource, round 1	2	Writer exiting

Readers-Writers Monitor Simulation

Monitor synchronization using a lock and condition variables. Events come from the Java execution.

Event 34 of 82

Shared resource: Being written

Show All Events | Play | Pause | Reset

Time (ms)	Thread	State	Event	Value	Resource Status
12	Reader-1	ENTER_READ	Entered reading section, round 1	0	Being read (1)
33	Reader-1	READING	Reading shared resource, round 1	0	Being read (1)
33	Reader-2	ENTER_READ	Entered reading section, round 1	0	Being read (2)
33	Reader-2	READING	Reading shared resource, round 1	0	Being read (2)
33	Reader-3	ENTER_READ	Entered reading section, round 1	0	Being read (3)
33	Reader-3	READING	Reading shared resource, round 1	0	Being read (3)
33	Reader-4	ENTER_READ	Entered reading section, round 1	0	Being read (4)
33	Reader-4	READING	Reading shared resource, round 1	0	Being read (4)
34	Writer-1	WAITING	Waiting for exclusive access, round 1	0	Being read (4)
34	Writer-2	WAITING	Waiting for exclusive access, round 1	0	Being read (4)
37	Reader-4	READING	Read value 0, round 1	0	Being read (4)
38	Reader-2	READING	Read value 0, round 1	0	Being read (4)
38	Reader-1	READING	Read value 0, round 1	0	Being read (4)
38	Reader-3	READING	Read value 0, round 1	0	Being read (4)
345	Reader-2	EXIT_READ	Finished reading; active readers: 3	0	Being read (3)
345	Reader-4	EXIT_READ	Finished reading; active readers: 2	0	Being read (2)
349	Reader-3	EXIT_READ	Finished reading; active readers: 1	0	Being read (1)
350	Reader-1	EXIT_READ	Finished reading; active readers: 0	0	Available
350	Writer-1	ENTER_WRITE	Entered writing section, round 1	0	Being written
351	Writer-1	WRITING	Updated value to 1, round 1	1	Being written
553	Reader-1	WAITING	Waiting to read, round 2	1	Being written
553	Reader-2	WAITING	Waiting to read, round 2	1	Being written
554	Reader-4	WAITING	Waiting to read, round 2	1	Being written
554	Reader-3	WAITING	Waiting to read, round 2	1	Being written
754	Writer-1	EXIT_WRITE	Finished writing, round 1	1	Writer exiting
754	Writer-1	RESOURCE_AVAILABLE	Writer released the shared resource	1	Available
754	Writer-2	ENTER_WRITE	Entered writing section, round 1	1	Being written
755	Writer-2	WRITING	Updated value to 2, round 1	2	Being written
1019	Writer-1	WAITING	Waiting for exclusive access, round 2	2	Being written
1159	Writer-2	EXIT_WRITE	Finished writing, round 1	2	Writer exiting
1159	Writer-2	RESOURCE_AVAILABLE	Writer released the shared resource	2	Available
1159	Writer-1	ENTER_WRITE	Entered writing section, round 2	2	Being written
1159	Writer-1	WRITING	Updated value to 3, round 2	3	Being written

3. DINING PHILOSOPHERS PROBLEM

Problem description

Four philosopher threads share four forks. Philosopher i uses fork i as the left fork and fork (i + 1) mod 4 as the right fork. A philosopher must acquire both forks before eating and release them afterwards. If every philosopher were allowed to hold one fork while waiting for another, circular waiting could cause deadlock.

Each philosopher completes three rounds. The console and event trace record thinking, becoming hungry, acquiring forks, eating, releasing forks, and finishing.

Semaphore-based implementation

The semaphore implementation creates four fork semaphores, each initialized with one permit. It also creates a room semaphore with N - 1 permits. With N = 4, at most three philosophers can enter the fork-acquisition section at the same time. This breaks the classic circular-wait scenario and prevents the standard deadlock configuration

Code Snippet 5. Fork and room semaphore declarations private
static final int N = 4; private static final int ROUNDS = 3;

private static final Semaphore[] forks = new Semaphore[N];
private static final Semaphore room = new Semaphore(N - 1);

static { for (int i = 0; i < N; i++) { forks[i] = new Semaphore(1); } }

Code Snippet 6. Acquiring and releasing forks room.acquire(); forks[left].acquire(); log(id, "HOLDING_FORK", "Picked up fork " + left);

forks[right].acquire(); log(id, "EATING", "Eating using forks " + left + " and " + right);

// After eating:

forks[right].release(); forks[left].release(); room.release();

This is a selected excerpt. The complete program includes timing delays, release-event messages, and interruption handling. The semaphores are created without an explicit fairness flag, so FIFO ordering is not guaranteed.

Monitor-style implementation

The Dining Philosophers monitor implementation uses Java's synchronized block, wait(), and notifyAll(). It does not use ReentrantLock or an array of Condition objects. A state array stores each philosopher as THINKING, HUNGRY, or EATING. The shared monitor object protects the state array.

When a philosopher becomes hungry, the program checks whether both neighbouring philosophers are not eating. If both neighbours are not eating, the philosopher changes to EATING. Otherwise, the philosopher waits. When eating finishes, the philosopher returns to THINKING, checks the neighbours, and notifies waiting threads so that they can recheck their conditions.

Code Snippet 7. Philosopher states and monitor declaration
private static final int N = 4; private static final int ROUNDS = 3;

private static final int THINKING = 0; private static final int
HUNGRY = 1; private static final int EATING = 2;

private static final int[] state = new int[N]; private static final Object
monitor = new Object();

Code Snippet 8. Checking whether a hungry philosopher can eat private

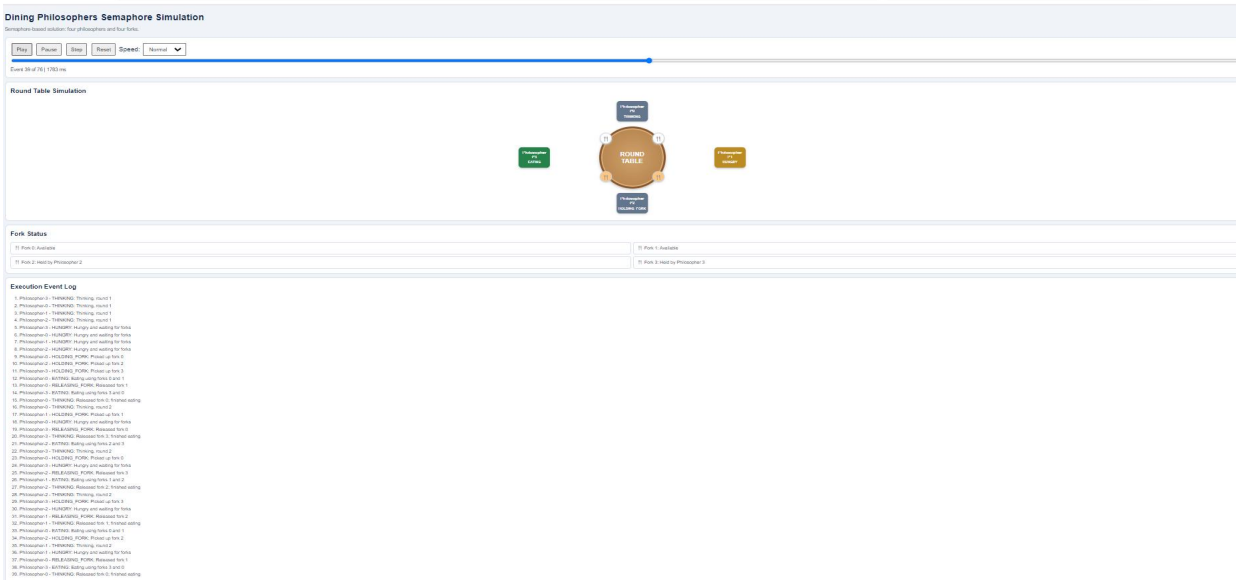
static void test(int id) { if (state[id] == HUNGRY && state[leftNeighbor(id)] != EATING && state[rightNeighbor(id)] != EATING) {

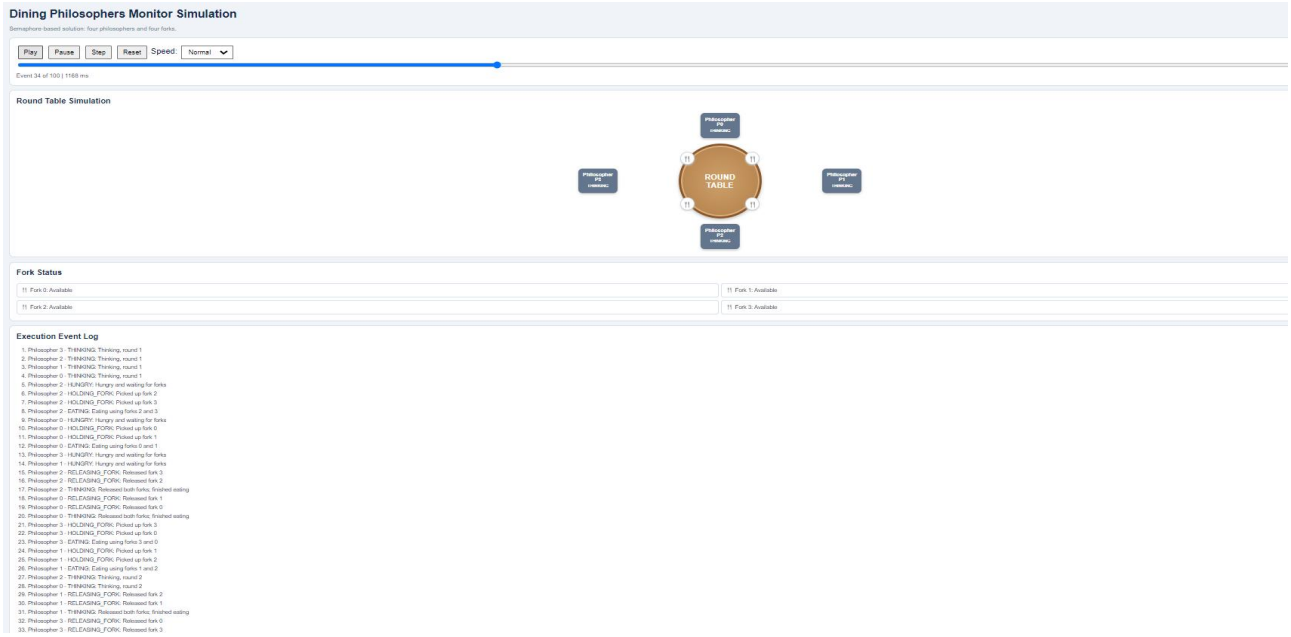
state[id] = EATING; monitor.notifyAll(); }

}

The complete program calls test() while holding the monitor lock inside a synchronized(monitor) block. This is necessary because notifyAll() must be called by a thread that owns that object's monitor. The algorithm prevents neighbouring philosophers from eating simultaneously; however, notifyAll() does not guarantee that every waiting philosopher will be scheduled promptly.Dining Philosophers HTML output

The simulation shows all four philosophers around a round table, the state of each philosopher, fork availability or ownership, playback controls, and an event log. The pictured run is a replay of recorded events, not the Java threads executing inside the browser.





4. EVENT RECORDING AND HTML SIMULATION

Event recording

TraceRecorder records important Dining Philosophers events with elapsed time, philosopher name, state, and a message. Fork details are included in the message text. The Readers-Writers event data also records the shared integer value and information about reader/writer activity

Code Snippet 9. Recording a Dining Philosophers event TraceRecorder.record(

"Philosopher-" + id, state, message);

This call passes the philosopher identifier, state, and message to TraceRecorder. The recorder stores each event in JSON form for later playback.

Generating the HTML file

The TraceRecorder creates the generated_html directory under the Java program's working directory, writes the event data, generates HTML containing the recorded events, and prints the absolute path of the HTML file. The output path printed by the Java program is the most reliable way to locate the generated page.

Code Snippet 10. Writing the HTML output String html = createHtml(title, json);

Path htmlFile = OUTPUT_DIR.resolve(htmlFileName); Files.writeString(

htmlFile, html, StandardCharsets.UTF_8);

System.out.println(

"HTML generated at: " + htmlFile.toAbsolutePath());

The Java-generated Dining Philosophers pages use the names DiningPhilosophersSimulation.html and DiningPhilosophersMonitorSimulation.html. Both are written under generated_html when their corresponding programs run.

Playback behaviour

The generated HTML pages provide playback controls such as Play, Pause, Step, Reset, and speed selection, depending on the page version. The round-table view represents philosophers and forks, while the event log lists the

actions in the recorded sequence. The Readers-Writers view displays thread states, shared-resource status, and shared values.

5. EXECUTION RESULTS AND OBSERVATIONS

Readers-Writers

The captured Readers-Writers simulation shows multiple readers entering the reading section together. Writers wait while readers are active. When the final reader exits, a writer can enter the exclusive writing section and update the shared value. Subsequent readers observe the updated value after writing finishes. The expected final value is 6 because two writers perform three increments each from an initial value of zero. This is the expected result from the code configuration and should be confirmed by the completed console run.Dining Philosophers

Dining Philosophers

The captured HTML event log includes philosophers moving from THINKING to HUNGRY, acquiring forks, entering EATING, and releasing forks. Fork-status indicators help show that a fork is either available or held by a philosopher. The exact order can change between runs because the thread scheduler determines when each thread executes. The semaphore solution limits the number of philosophers competing to acquire forks to three. The monitor solution checks the states of neighbouring philosophers before granting permission to eat. Both approaches are designed to prevent the classic deadlock scenario, but starvation freedom is not guaranteed by the current implementations under every possible scheduleTest summary

Test summary

Test	Expected behaviour	Evidence to verify
Readers-Writers semaphore	No reader/writer overlap; writers update exclusively	Console completion and final value 6
Readers-Writers monitor	Writer priority while waiting writers exist	Console trace and monitor HTML
Dining Philosophers semaphore	Each philosopher acquires both forks before eating	Console trace and semaphore HTML
Dining Philosophers monitor	Only philosophers with non-eating neighbours may eat	Console trace and monitor HTML
Event generation	HTML reflects recorded execution events	Printed output paths and browser pages

6. SYNCHRONIZATION ANALYSIS

Mutual exclusion

In Readers-Writers, resourceLock prevents writers from modifying the shared value while readers are using it. The monitor implementation protects the shared state using ReentrantLock and controls entry with conditions. In Dining Philosophers, each fork semaphore has one permit, so a fork cannot be acquired by two philosophers simultaneously. The monitor implementation uses a protected state array and permits a philosopher to eat only when both neighbours are not eatingDeadlock prevention

Deadlock prevention

Deadlock requires a set of threads to wait indefinitely for resources held by one another. In the semaphore-based Dining Philosophers solution, the room semaphore permits only N - 1 philosophers to compete for forks, breaking the classic circular-wait pattern. In the monitor solution, a philosopher does not acquire one fork and then wait while holding it; the state test grants permission to eat only when both neighbours are not eating.

Starvation and fairness

The Readers-Writers monitor uses a fair ReentrantLock and writer-priority logic, which reduces the risk of writers being delayed by a continuous stream of readers. However, fairness at the lock level does not by itself prove every possible scheduling outcome. The Dining Philosophers semaphores do not set the fairness argument to true, and notifyAll() does not select a specific waiting philosopher. Therefore, the report does not claim a strict starvation-freedom guarantee.

Race conditions and scheduling

A race condition can occur if threads update shared state without synchronization. Protecting the active reader count, shared value, philosopher state array, and fork acquisition prevents the conflicting operations targeted by these solutions. Console and HTML event order may differ across runs because concurrent threads are scheduled nondeterministically.

7.COMPARISON OF APPROACHES

Feature	Semaphore approach	Monitor-style approach
Readers-Writers primitives	readerMutex and resourceLock	ReentrantLock and Condition
Dining Philosophers primitives	Four fork semaphores and room semaphore	synchronized, wait(), notifyAll()
Resource control	Explicit acquire() and release() permits	Protected state and condition checks
Deadlock strategy	Limit competing philosophers to N - 1	Allow eating only when both neighbours are not eating
Fairness	Depends on semaphore configuration	Depends on lock policy and thread scheduling
Common implementation risk	Incorrect permit release or acquisition order	Incorrect state protection or notification

Both approaches can coordinate concurrent threads correctly when the synchronization rules are implemented consistently. Semaphores provide direct control over permits. Monitor-style implementations make the protected state and the conditions for entering a critical section explicit.

8LIMITATIONS AND POSSIBLE IMPROVEMENTS

1. Thread counts and rounds are fixed in the current programs.
2. Thread scheduling may change the order of recorded events.
3. Dining Philosophers semaphores do not explicitly guarantee fair ordering.
4. notifyAll() does not guarantee which waiting thread proceeds first.
5. HTML animations replay recorded events rather than performing synchronization.
6. The monitor HTML subtitle should correctly identify the monitor implementation.

Possible improvements include configurable thread counts, fairness controls, automated trace validation, and accurate HTML labels

9.CONCLUSION

This assignment implemented the Readers-Writers and Dining Philosophers problems in Java using semaphore-based and monitor-style synchronization. The Readers-Writers semaphore solution uses readerMutex and resourceLock, while its monitor-style solution uses a fair ReentrantLock, Condition variables, and writer-priority logic. The Dining Philosophers semaphore solution uses four fork semaphores and a room semaphore to prevent the classic deadlock scenario. The monitor solution uses synchronized, wait(), notifyAll(), and a state array to control permission to eat. Event recording and HTML playback provide a useful way to observe concurrent execution. The recorded traces demonstrate how readers share access, writers update shared data exclusively, and philosophers coordinate fork use. The implementation also illustrates why deadlock prevention, fairness, and starvation must be considered separately when designing synchronization algorithms

REFERENCES

The following eight textbooks cover operating systems, concurrency, synchronization primitives, and concurrent programming.

[1] Silberschatz, A., Galvin, P. B., and Gagne, G. Operating System Concepts. 10th ed., Wiley, 2018.

[2] Tanenbaum, A. S., and Bos, H. Modern Operating Systems. 4th ed., Pearson, 2015.

[3] Stallings, W. Operating Systems: Internals and Design Principles. 9th ed., Pearson, 2018.

[4] Deitel, P. J., and Deitel, H. M. Java How to Program. 11th ed., Pearson, 2018.

[5] Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., and Lea, D. Java Concurrency in Practice. Addison-Wesley, 2006

. [6] Lea, D. Concurrent Programming in Java: Design Principles and Patterns. 2nd ed., Addison-Wesley, 2000.

[7] Ben-Ari, M. Principles of Concurrent and Distributed Programming. 2nd ed., Addison-Wesley, 2006

. [8] Herlihy, M., and Shavit, N. The Art of Multiprocessor Programming. Revised reprint, Morgan Kaufmann, 2012..

AI Usage Summary

AI tools were used to assist with Java code development, troubleshooting, HTML simulation generation, and report preparation. The student reviewed the code, addressed implementation issues, and checked the program outputs. A limitation identified during development was the need to correct HTML output and file-opening behaviour

DECLARATION

I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own.

Name: Devika K

USN: NNM24IS065

Signature: _____

Date: _____